Python Course for Physicists (And everyone else!)

Chunk 8: 1D Numpy Arrays

By Jonathan Bollig

09.10.2025

Numpy: Numerical Python;
→Lists but much better

- Problem: loops and lists are “slow”:
  → Example: square 100 000 000 numbers and sum them
  → 28 seconds

```
# list_data = [1, 2, 3, ...., 1e8]
start = time()

list_new_data = []
for i in list_data:
    list_new_data.append(i**2)
list_squared_sum = sum(list_new_data)

print("Time for list: " + str(time()-start))
```

```
Time for list: 28.533897638320923
```

- Solution: Using C-code in python
  → 0.5 seconds
  → Numpy is typically 10 – 100 x faster

```
start = time()

np_squared_sum = np.sum(np_data**2)

print("Time for numpy: " + str(time()-start))
```

```
Time for numpy: 0.46373486518859863
```

- Bonus: Cleaner, easier to read code

Live Coding: Creating Arrays

- 1. Step: import numpy as np
- Converting “list → array”:
  np.array():
- Converting “array → list”:
  array.tolist() :
- Careful! Only one data type:
- Nested lists: Only when all elements have lists of same length.

Convention!

```
import numpy as np

my_list = [1, 2, 3, 4, 5]
print(my_list)
my_array = np.array(my_list)
print(my_array)
new_list = my_array.tolist()
print(new_list)
```

```
[1, 2, 3, 4, 5]
[1 2 3 4 5]
[1, 2, 3, 4, 5]
```

```
mixed_list = [3, 2.4, "hi"]
mixed_array = np.array(mixed_list)
print(mixed_array)
```

```
['3' '2.4' 'hi']
```

Live Coding: Behavior – Similarities

- Printing and slicing behave the same

```
print(my_list)
print(my_array)
```

```
[1, 2, 3, 4, 5]
[1 2 3 4 5]
```

```
print(my_list[0])
print(my_array[0])
```

```
1
1
```

```
print(my_list[2:])
print(my_array[2:])
```

```
[3, 4, 5]
[3 4 5]
```

- Setting values stays the same:

```
my_list[0] = 5
print(my_list)

my_array[0] = 5
print(my_array)
```

```
[5, 2, 3, 4, 5]
[5 2 3 4 5]
```

Live Coding: Behavior – Similarities

- Looping:
  Be warned: Looping is slow

```
for i in my_list:
    print(i)
print()
for i in my_array:
    print(i)
```

```
5
2
3
4
5

5
2
3
4
5
```

Live Coding: Behavior - Differences

- Adding lists:

```
print(my_list + my_list)
print(my_array + my_array)
```

```
[5, 2, 3, 4, 5, 5, 2, 3, 4, 5]
[10  4  6  8 10]
```

- Multiplying lists:

```
print(3 * my_list)
print(3 * my_array)
```

```
[5, 2, 3, 4, 5, 5, 2, 3, 4, 5, 5, 2, 3, 4, 5]
[15  6  9 12 15]
```

- (Almost ?) All mathematical operators work element-wise!
  → Lists need same shape!

```
print(my_array**2)
print(my_array % 2)
print(my_array / 10)
```

```
[25  4  9 16 25]
[1 0 1 0 1]
[0.5 0.2 0.3 0.4 0.5]
```

Live Coding: Array Generation Functions

- Arrays can be created by other means than passing a list
- Direct ways:
  - `np.arange(start, stop, step)`: [start, start + step, start + 2* step, …] until you reach stop (excluding)
  - `np.linspace(start, stop, n)`: Array of length n from start to stop, equally Spaced
  - `np.zeros(n)` / `np.ones(n)`: Array filled with 0 / 1 of length n.
  - `np.full(n, what)`: Array of length n, filled with „what“

Good Luck with your Tasks ☺

[https://www.reddit.com/r/ProgrammerHumor/comments/g6z7qm/no_problem/]